

WEEK 12 — Unit 4: Maven Build Automation

Topic: Maven and Docker Integration — Dockerizing Maven Applications & Pushing Artifacts to Registries
Course: INT 332 – DevOps

Note 1: Why Integrate Maven with Docker

The Gap Between Build and Deployment

By the end of Week 11, we know that Maven can compile code, run tests, and package a Java application into a JAR or WAR file. However, that JAR file on its own is not enough for modern deployment. To deploy a Java application consistently across different environments — development, staging, production — you need to package it into a Docker image.

Without integration between Maven and Docker, the typical workflow looks like this:

- Developer runs `mvn package` to produce the JAR
- Developer manually writes or maintains a Dockerfile
- Developer manually runs `docker build` to create the image
- Developer manually runs `docker push` to push to a registry

This is a multi-step, error-prone process. If the developer forgets to rebuild the Docker image after a code change, the deployed container may be running stale code. Integrating Docker into the Maven build lifecycle solves this by making Docker image creation and pushing an automatic part of the Maven build.

Goals of Maven-Docker Integration

- Build a Docker image automatically as part of `mvn package` or `mvn install`
- Tag the image consistently using the Maven project version
- Push the image to Docker Hub or another registry as part of `mvn deploy`
- Keep the entire build pipeline — from source code to deployable image — in a single command

Overview of Available Plugins

Several Maven plugins support Docker integration. The two most widely used are:

Plugin	GroupId	Best Used For
dockerfile-maven-plugin	com.spotify	Simple Dockerfile-based builds
docker-maven-plugin	io.fabric8	Full-featured, no separate Dockerfile needed
jib-maven-plugin	com.google.cloud.tools	Daemonless builds, no Docker required locally

This week focuses primarily on the **dockerfile-maven-plugin** by Spotify and the general approach to Dockerizing Maven applications, since these are the most straightforward to understand and the most commonly referenced in DevOps courses.
